IFT605 / IFT713 — Systèmes Répartis et Multi-Agents

Université de Sherbrooke — Automne 2026

Rapport de projet

SmartAgent-IoT

Plateforme domotique multi-agents avec MQTT, openHAB et Java RMI

Équipe : lyzc3821

Enseignante : Amal Ben Abdellah

Date : 27 octobre 2026

1. Introduction et mise en contexte

Ce projet implémente SmartAgent-IoT, une plateforme domotique simulant un étage de bâtiment intelligent composé de trois pièces (salleA, salleB, cuisine). Chaque pièce est équipée de trois capteurs simulés (température, luminosité, occupation) et d'un actuateur (thermostat).

L'architecture combine trois technologies aux rôles complémentaires et volontairement différents dans leur modèle de communication :

- MQTT (Eclipse Mosquitto) : communication asynchrone publish/subscribe entre les capteurs et les agents — les capteurs ignorent complètement qui consomme leurs données.
- Java RMI : communication synchrone par appel de méthode distant entre les agents de pièce et le coordinateur central, dans des JVM séparées.
- openHAB 4 : couche d'intégration et tableau de bord, qui consomme les mêmes flux MQTT et exécute des règles d'automatisation indépendantes des agents Java.

Ce contraste délibéré entre pub/sub découplé et appel de méthode synchrone est le cœur pédagogique du cours : la Section 7.4 documente un cas concret où ces deux modèles sont entrés en compétition sur la même donnée.

2. Architecture générale

Le projet est structuré en un dépôt Maven multi-module, reflétant directement les trois rôles fonctionnels :

```
smartagent-iot/
├── docker-compose.yml       (Mosquitto + openHAB, ports publiés 1883/8080)
├── mosquitto/config/mosquitto.conf
├── openhab/conf/
│   ├── things/batiment.things   items/batiment.items   rules/batiment.rules
│   ├── sitemaps/batiment.sitemap  (tableau de bord Basic UI, voir 3.5)
│   └── services/addons.cfg      (installe mqtt + ui=basic automatiquement)
├── sensor-simulator/        (module Maven : capteurs -> MQTT)
├── agent-piece/           (module Maven : IAgentPiece + AgentPiece, RMI+MQTT)
└── coordinateur/           (module Maven : CoordinateurBatiment, RMI)
```

Ports et conventions de nommage utilisés dans toute l'implémentation :

Composant	Port / convention	Rôle
Broker MQTT (Mosquitto)	1883	Transport pub/sub des mesures et commandes
Topics capteurs	batiment/{piece}/capteur/{type}	temperature, luminosite, occupation
Topics actuateurs	batiment/{piece}/actuateur/thermostat	Consigne envoyée par un AgentPiece ou une règle DSL
Registre RMI	2199	Annuaire partagé des 3 AgentPiece (agent-salleA, agent-salleB, agent-cuisine)
openHAB (UI + REST)	8080	Tableau de bord et vérification de l'état via /rest/items

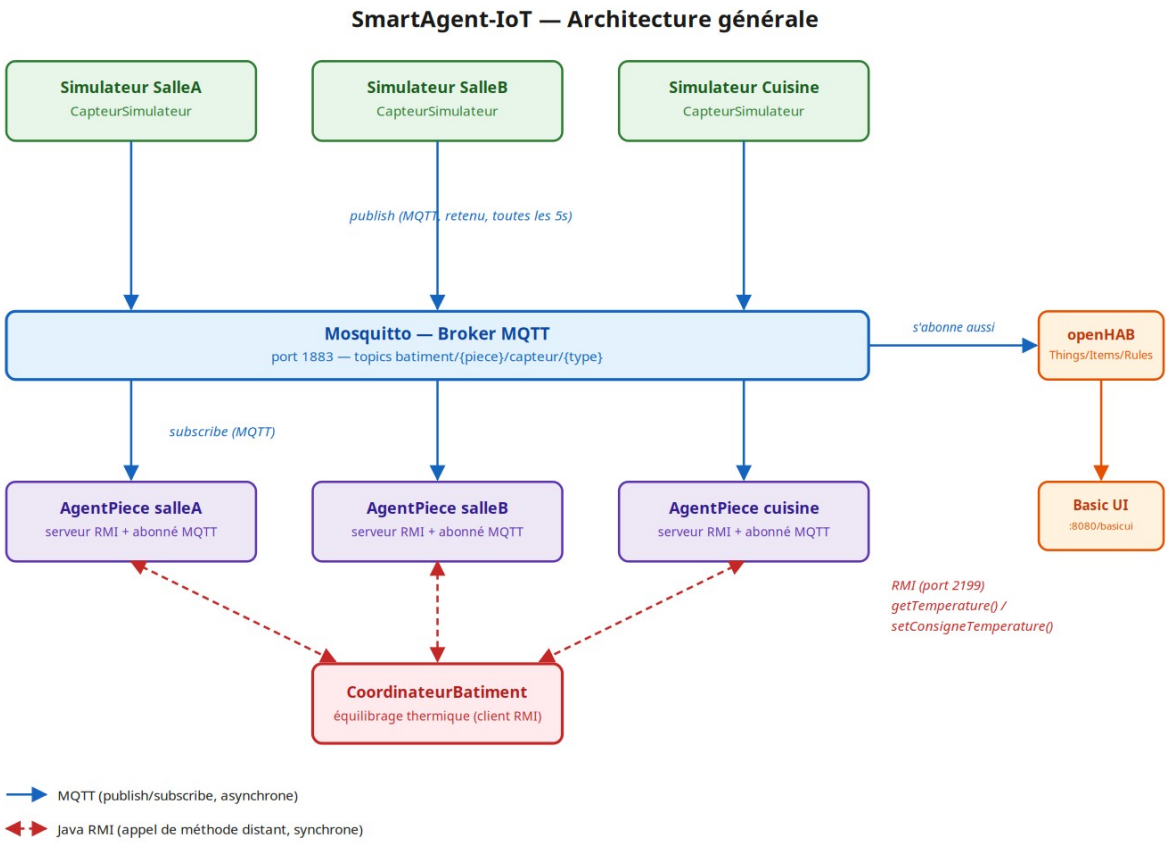


Figure 1 — Architecture generale de SmartAgent-IoT (capteurs, broker MQTT, agents RMI, coordinateur, openHAB)

2.1 Flux de données

Deux chemins de données distincts coexistent et convergent sur les mêmes topics MQTT :

- Chemin de lecture : CapteurSimulateur → MQTT (retained) → AgentPiece (abonné) → CoordinateurBatiment (RMI) et → openHAB (Thing MQTT) → Items → Règles DSL.
- Chemin de commande : CoordinateurBatiment.setConsigneTemperature() (RMI) → AgentPiece → MQTT (topic actuateur) → openHAB Item, EN PARALLÈLE des règles DSL openHAB qui publient directement sur le même topic actuateur (ex. économie d'énergie).

Cette convergence de deux sources de commande indépendantes sur le même topic actuateur est un choix assumé de l'énoncé (le thermostat peut être piloté par le coordinateur ET par openHAB) ; voir la Section 7.4 pour un exemple concret observé.

3. Choix de conception

3.1 Simulateurs de capteurs (Tâche 1.1)

Plages de valeurs documentées et justifiées :

Capteur	Plage	Justification
Température	17.0 – 26.0 °C	Plage de confort intérieur réaliste avec marge basse/haute

Luminosité	0 – 800 lux	Pièce sombre à très éclairée
Occupation	40% de probabilité	Une pièce vide plus souvent qu'occupée, sans être irréaliste

La température évolue par une marche aléatoire bornée (±0.5°C par cycle de 5s) plutôt qu'un tirage uniforme indépendant à chaque publication : un vrai capteur ne peut pas sauter de 18°C à 25°C entre deux mesures consécutives. Chaque message est publié avec le drapeau retained=true (le broker conserve la dernière valeur) afin qu'un abonné qui démarre après le simulateur — typiquement openHAB ou un agent redémarré — reçoive immédiatement le dernier état connu au lieu d'attendre jusqu'à 5 secondes.

3.2 Contrat RMI et gestion de la concurrence (Tâche 2.1 / 2.2)

Chaque méthode de IAgentPiece déclare RemoteException, y compris celles qui ne l'utilisent pas en interne : un appel RMI traverse le réseau et peut échouer à tout moment, contrairement à un appel local qui ne peut échouer que par un bug du code appelé.

Les champs temperature, luminosite et occupation dans AgentPiece sont déclarés volatile. Ils sont écrits par le thread de callback MQTT (interne à la librairie Paho) et lus par les threads RMI qui traitent les appels distants entrants — deux threads distincts. Sans volatile, un thread RMI pourrait lire une valeur mise en cache dans son propre cœur de CPU plutôt que la dernière valeur réellement écrite (problème classique de visibilité mémoire en Java, résolu ici sans verrou explicite puisqu'il s'agit de lectures/écritures simples, pas d'opérations composées).

Le registre RMI (port 2199) est un service unique partagé par les 3 agents, démarrés comme 3 processus JVM indépendants. Comme l'ordre de démarrage n'est pas garanti, chaque AgentPieceMain tente de CRÉER le registre et, si le port est déjà occupé (ExportException), se CONNECTE simplement au registre existant à la place.

3.3 Politique d'équilibrage du coordinateur (Tâche 2.3)

Lorsque l'écart de température entre deux pièces adjacentes dépasse 4°C, la consigne envoyée à la pièce la plus chaude est la MOYENNE des deux températures, plutôt qu'un alignement direct sur la température de la pièce la plus fraîche. Un alignement brutal (ex. 28°C → 20°C d'un coup) imposerait un choc thermique irréaliste ; converger vers la moyenne rapproche progressivement les deux pièces, et le prochain cycle d'équilibrage (toutes les 10s) affine si l'écart persiste.

Chaque paire adjacente est traitée dans son propre bloc try/catch : une RemoteException sur une paire (agent injoignable) est journalisée puis le cycle continue avec les paires suivantes, au lieu d'interrompre tout le processus d'équilibrage.

3.4 Timeout et tolérance aux pannes (Tâche 3.1)

Un capteur est considéré en timeout après 30 secondes sans nouveau message MQTT (seuil imposé par l'énoncé). Deux mécanismes indépendants et complémentaires ont été mis en place :

- Détection applicative : un thread "watchdog" dans chaque AgentPiece vérifie toutes les 5 secondes l'âge de la dernière donnée reçue. La transition panne→détection et panne→reprise n'est journalisée qu'une seule fois chacune (et non à chaque vérification), grâce à un drapeau booléen (timeoutDejaSignale).
- Affichage UNDEF côté openHAB : plutôt que d'écrire du code Java pour forcer un état UNDEF, un profil natif "expire" a été appliqué directement sur les Items liés aux capteurs (pas au thermostat, qui est une consigne que le système envoie lui-même). C'est la manière idiomatique openHAB de répondre à cette exigence, sans dupliquer la logique de timeout déjà présente côté agent.

Le résultat est qu'un agent qui ne reçoit plus de données NE BLOQUE JAMAIS ses méthodes RMI : getTemperature() continue de répondre instantanément avec la dernière valeur connue, exactement comme l'exige l'énoncé ("le coordinateur détecte le timeout et passe à la dernière valeur connue").

3.5 Tableau de bord : Basic UI plutôt que Main UI (Tâche 1.2)

openHAB 4 propose deux interfaces web distinctes. La "Main UI" (interface par défaut sur http://localhost:8080/) est une application moderne où les pages et widgets se construisent manuellement dans le navigateur, ou automatiquement si les Items portent des tags sémantiques (Location/Equipment) — ce qui n'est pas notre cas ici. La "Basic UI" génère au contraire un tableau de bord immédiatement à partir d'un simple fichier texte, le sitemap.

Basic UI a été choisie pour rester cohérente avec le reste du projet : toute la configuration openHAB (things, items, rules, et maintenant sitemap) est versionnée en fichiers texte dans conf/, sans aucune étape de configuration manuelle dans le navigateur. Le binding ui=basic est activé automatiquement au démarrage via addons.cfg, exactement comme le binding mqtt. Le tableau de bord est accessible à http://localhost:8080/basicui/app?sitemap=batiment.

3.6 Consultation pair-à-pair entre agents via RMI (objectif S2)

L'énoncé demande, en Semaine 2, de « permettre aux agents de se consulter mutuellement via RMI (partage d'état) ». Nous distinguons donc deux topologies de coordination, volontairement complémentaires : une coordination centralisée, où le CoordinateurBatiment interroge tous les agents et arbitre (Section 3.3) ; et une coordination pair-à-pair, où chaque AgentPiece interroge directement ses voisins, sans intermédiaire.

Ce choix illustre que RMI n'impose aucune hiérarchie : un AgentPiece est à la fois serveur RMI (il expose IAgentPiece au coordinateur) et client RMI (il appelle getTemperature() sur ses voisins). C'est exactement la dualité d'un système multi-agents réparti.

Conception retenue :

- Le voisinage suit l'adjacence imposée par l'énoncé (salleA–salleB, salleB–cuisine). salleB, au centre, a deux voisins ; salleA et cuisine en ont un. La table est fournie à l'agent au démarrage par AgentPieceMain, et non codée en dur dans AgentPiece : l'agent reste agnostique de la topologie.
- Un thread démon dédié (consultation-<piece>) effectue, toutes les 10 s, un lookup de chaque voisin dans le registre puis lit sa température en direct. Il est séparé du watchdog de timeout et des appels RMI entrants : la consultation ne bloque jamais la disponibilité de l'agent.
- Le lookup est refait à chaque cycle (et non mémorisé) : un voisin peut démarrer après nous, ou tomber puis revenir ; cela rend la consultation robuste à l'ordre de démarrage et aux pannes.
- Partage d'état : la dernière température connue de chaque voisin est conservée dans une ConcurrentHashMap (écrite par le thread de consultation, lue par les threads RMI) et exposée dans getEtatJson() sous la clé « voisins ».
- Tolérance aux pannes (cohérente avec la Tâche 3.1) : un voisin non encore enregistré lève NotBoundException, un voisin mort lève RemoteException ; dans les deux cas on ignore ce voisin pour ce cycle seulement, sans interrompre la consultation des autres.

Extrait (cœur de la boucle de consultation, AgentPiece.java) :

```
private void consulterUnVoisin(String voisin) {
    try {
        Registry registre = LocateRegistry.getRegistry("localhost", PORT_REGISTRE);
        IAgentPiece stubVoisin = (IAgentPiece) registre.lookup("agent-" + voisin);
        double tempVoisin = stubVoisin.getTemperature();  // appel RMI agent -> agent
        dernieresTempVoisins.put(voisin, tempVoisin);      // partage d'etat (thread-safe)
    } catch (NotBoundException e) {
        dernieresTempVoisins.remove(voisin);  // voisin pas encore demarre : normal
    } catch (RemoteException e) {
        dernieresTempVoisins.remove(voisin);  // voisin tombe : on continue
    }
}
```

3.7 Intégration coordinateur → règles openHAB via l'API REST (objectif S3)

L'énoncé demande, en Semaine 3, d'« intégrer le coordinateur RMI avec les règles openHAB (via l'API REST openHAB) ». Il existe désormais deux chemins par lesquels une décision du coordinateur atteint openHAB. Le chemin indirect RMI → MQTT (déjà présent) : le coordinateur appelle setConsigneTemperature() sur l'agent, qui publie la consigne sur batiment/{piece}/actuateur/thermostat, lue par openHAB via son canal MQTT. Le chemin direct REST (ajout S3) : le coordinateur envoie lui-même une commande HTTP à un Item openHAB, ce qui déclenche une règle DSL.

Le second chemin apporte ce que le premier ne fait pas : il relie explicitement une décision d'arbitrage du coordinateur au moteur de règles d'openHAB et au tableau de bord, sans passer par le bus de mesures des capteurs.

Conception retenue :

– Nouvelle classe OpenHabRestClient (module coordinateur) bâtie sur java.net.http.HttpClient, standard du JDK depuis Java 11 : aucune dépendance Maven ajoutée pour un simple POST de texte.

– Contrat utilisé : POST /rest/items/{item} avec un corps text/plain envoie une commande à l'Item (déclenche les règles « changed »/« received command »), à la différence de PUT .../state qui ne ferait que forcer l'état.

– À chaque déséquilibre traité, le coordinateur pousse un résumé de sa décision sur un Item dédié Batiment_Coordination (un String sans canal MQTT, alimenté uniquement par le coordinateur). La règle DSL « Coordination batiment » réagit à son changement : c'est le pont coordinateur ↔ règles openHAB demandé. L'Item est aussi affiché dans le sitemap.

– Best-effort : si openHAB est absent ou répond en erreur, envoyerCommande() journalise et renvoie false sans jamais propager d'exception ni interrompre le cycle d'équilibrage (même exigence de tolérance que la Tâche 3.1). L'intégration est injectable et optionnelle : CoordinateurBatiment accepte un OpenHabRestClient pouvant être null (test de la politique pure sans openHAB), et l'URL est configurable par -Dopenhab.url=... (défaut http://localhost:8080).

Extrait côté coordinateur (CoordinateurBatiment.traiterPaire) :

```
agentChaud.setConsigneTemperature(consigneCible);     // chemin 1 : RMI -> MQTT
if (openhab != null) {                                // chemin 2 : REST -> regle openHAB
   String decision = String.format(Locale.US,
      "%s: consigne %.1fC (desequilibre %s/%s, ecart %.1fC)",
      nomChaud, consigneCible, nomA, nomB, ecart);
   openhab.envoyerCommande("Batiment_Coordination", decision);
}
```

Règle DSL correspondante (batiment.rules) :

```
rule "Coordination batiment"
when
   Item Batiment_Coordination changed
then
   logInfo("SmartAgentIoT", "Coordinateur (via REST): " + Batiment_Coordination.state)
end
```

4. Implémentation détaillée par semaine

4.1 Semaine 1 — Infrastructure IoT et MQTT

- Mosquitto déployé via Docker Compose (ports 1883/8080 publiés ; openHAB partage le réseau du broker via network_mode: service:mosquitto, pour que host="localhost" reste valide pour les processus Java hors conteneur comme pour openHAB).
- Module sensor-simulator : CapteurSimulateur publie les 9 flux (3 pièces × 3 mesures) toutes les 5 secondes.

- openHAB configuré par fichiers texte (batiment.things, batiment.items, batiment.rules, batiment.sitemap) plutôt que par l'interface graphique, pour que la configuration soit versionnée avec le code.
- 2 règles DSL fonctionnelles : économie d'énergie (consigne à 18°C quand une pièce se vide) et alerte (température hors de la plage 15-27°C).
- Tableau de bord Basic UI (voir 3.5) généré à partir de batiment.sitemap, accessible sur http://localhost:8080/basicui/app?sitemap=batiment.

Sortie console réelle (validée le 2026-10-27) — état des Items openHAB (ce que le tableau de bord Basic UI affiche) :

```
$ curl -s "http://localhost:8080/rest/items?fields=name,state"
SalleA_Temperature  20.3    SalleA_Luminosite  198   SalleA_Occupation  OFF
SalleB_Temperature  22.0    SalleB_Luminosite  252   SalleB_Occupation  OFF
Cuisine_Temperature 21.2    Cuisine_Luminosite  34   Cuisine_Occupation ON
SalleA_Thermostat 21.0   SalleB_Thermostat 18.0   Cuisine_Thermostat 18.0
(URL du tableau de bord : http://localhost:8080/basicui/app?sitemap=batiment)
```

Sortie console réelle (validée le 2026-10-27) — les 9 flux de capteurs (3 pièces x 3 mesures) :

```
$ mosquitto_sub -h localhost -t 'batiment/#' -v | sort -u
batiment/salleA/capteur/temperature 19.3
batiment/salleA/capteur/luminosite 175
batiment/salleA/capteur/occupation true
batiment/salleB/capteur/temperature 21.4
batiment/salleB/capteur/luminosite 146
batiment/salleB/capteur/occupation true
batiment/cuisine/capteur/temperature 17.0
batiment/cuisine/capteur/luminosite 416
batiment/cuisine/capteur/occupation false
```

4.2 Semaine 2 — Agents distribués Java RMI

- IAgentPiece : interface distante avec les 5 méthodes du contrat (voir 3.2).
- AgentPiece : serveur RMI + abonné MQTT par pièce, avec gestion thread-safe via des champs volatile.
- CoordinateurBatiment : politique d'équilibrage thermique sur les paires adjacentes salleA-salleB et salleB-cuisine.
- 3 agents lancés comme 3 processus JVM indépendants, communiquant exclusivement via le registre RMI partagé (port 2199) — fidèle à l'esprit "appels de méthodes distants entre JVM" de l'énoncé.

Sortie console réelle (validée le 2026-10-27) — démarrage des 3 agents (1 crée le registre RMI, 2 s'y connectent) :

```
Registre RMI créé sur le port 2199
[agent-salleA] abonné à batiment/salleA/capteur/#
AgentPiece[salleA] enregistré sous 'agent-salleA' dans le registre RMI (localhost:2199)
Registre RMI déjà actif sur le port 2199, connexion...
AgentPiece[salleB] enregistré sous 'agent-salleB' dans le registre RMI (localhost:2199)
Registre RMI déjà actif sur le port 2199, connexion...
AgentPiece[cuisine] enregistré sous 'agent-cuisine' dans le registre RMI (localhost:2199)
```

Sortie console réelle (validée le 2026-10-27) — lecture d'état + envoi de consigne, depuis une AUTRE JVM, via IAgentPiece :

```
$ java -cp agent-piece/target/agent-piece.jar \
    ca.udes.ift605.smartagent.agent.AgentPieceTestClient salleA 21.0
=== Lecture de l'état de salleA via RMI ===
Température : 20.3
Luminosité  : 198
Occupée     : false
getEtatJson : {"enTimeout":false,"dernierMessage":"2026-10-27T22:04:42.186Z",
              "piece":"salleA","luminosite":198,"occupee":false,"temperature":20.3}
=== Envoi d'une nouvelle consigne (21.0) via RMI ===
Commande envoyée avec succès (vérifiable ensuite sur MQTT/openHAB).
```

4.3 Semaine 3 — Intégration, tolérance aux pannes et tests

- Extension du contrat RMI avec estEnTimeout(), sans modifier les 5 méthodes d'origine.
- Watchdog de 30s dans AgentPiece + profil expire côté openHAB.
- Simulateurs rendus isolables par pièce (un processus par pièce) pour pouvoir couper une seule pièce sans affecter les deux autres — nécessaire pour un test de panne réaliste.
- 4 tests JUnit sur la politique d'équilibrage du coordinateur (voir 5.3).

5. Tests et validation

5.1 Validation MQTT / openHAB (Semaine 1)

Les 9 flux ont été vérifiés séparément avec mosquitto_sub, et les mêmes valeurs ont été confirmées visibles dans l'interface REST d'openHAB (GET /rest/items). Les deux règles DSL ont été déclenchées de façon reproductible :

- Règle d'économie d'énergie : déclenchée automatiquement dès qu'une occupation passe à OFF, confirmée par le journal openHAB et par le changement de l'Item thermostat correspondant à 18°C.
- Règle d'alerte : déclenchée en publiant manuellement une température de 31.5°C sur le topic d'une pièce, confirmée par le message de journal "ALERTE: SalleA hors plage (31.5 degC)".

Sortie console réelle (validée le 2026-10-27) — déclenchement des règles DSL (économie d'énergie + alerte) :

```
2026-10-27 18:10:27 [INFO] SmartAgentIoT - Economie d'energie: SalleB vide -> consigne 18 degC
2026-10-27 18:10:52 [INFO] SmartAgentIoT - Economie d'energie: Cuisine vide -> consigne 18 degC
2026-10-27 18:11:05 [WARN] SmartAgentIoT - ALERTE: Cuisine hors plage (30.0 degC)
```

5.2 Validation RMI et coordinateur (Semaine 2)

Un écart de température supérieur au seuil a été forcé volontairement entre salleA et salleB (28.5°C contre 20.7°C, écart de 7.8°C). Le coordinateur a calculé une consigne de 24.6°C (moyenne) et l'a envoyée à salleA via RMI ; la propagation jusqu'à MQTT puis openHAB a été confirmée par lecture de l'Item SalleA_Thermostat.

Sortie console réelle (validée le 2026-10-27) — cycle d'équilibrage : rien à faire sous le seuil, puis déséquilibre forcé :

```
[Coordinateur] salleA(19.7) / salleB(21.6) -> écart 1.9 (seuil 4.0), rien à faire
[Coordinateur] salleB(21.6) / cuisine(20.8) -> écart 0.8 (seuil 4.0), rien à faire
[Coordinateur] DESEQUILIBRE salleA(29.0) / salleB(21.4) -> écart 7.6 > seuil 4.0 :
               consigne 25.2 envoyée à salleA
```

5.3 Tests unitaires JUnit

Plutôt que de tester RMI et MQTT en direct (ce qui transformerait chaque test en test d'intégration lent et fragile, dépendant de ports libres et de processus externes), la LOGIQUE de décision du coordinateur a été isolée et testée avec un faux agent (FauxAgentPiece implémentant IAgentPiece sans réseau). Quatre cas sont couverts :

- Aucun ajustement si l'écart est sous le seuil.
- Ajustement correct (moyenne) si l'écart dépasse le seuil, uniquement sur la pièce la plus chaude.
- Cas limite : écart exactement égal à 4.0°C ne déclenche pas d'ajustement (seuil strictement supérieur).
- Un agent injoignable (RemoteException simulée) n'empêche pas le traitement des autres paires — validation directe de la tolérance aux pannes.

Résultat : 4/4 tests réussis (mvn clean test).

Sortie console réelle (validée le 2026-10-27) — tests unitaires de la politique du coordinateur :

```
$ mvn clean package
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]   -- in ca.udes.ift605.smartagent.coordinateur.CoordinateurBatimentTest
[INFO] BUILD SUCCESS
```

5.4 Validation des extensions S2 et S3

S2 — consultation agent ↔ agent (observée en exécution réelle). Les logs des agents montrent les appels RMI de pièce à pièce, et getEtatJson() expose l'état partagé des voisins :

```
[agent-salleA] consultation RMI de salleB : 20.0°C (ma temp 17.0°C, ecart 3.0°C)
[agent-salleB] consultation RMI de cuisine : 21.1°C (ma temp 20.0°C, ecart 1.1°C)
getEtatJson salleB : {"piece":"salleB","temperature":20.2,...,
            "voisins":{"salleA":20.7,"cuisine":23.6}}
```

[CAPTURE : terminaux des agents salleA/salleB montrant les lignes « consultation RMI de … »]

S3 — notification REST vers openHAB (observée de bout en bout). Lors d'un déséquilibre réel, le coordinateur poste sa décision sur openHAB et reçoit HTTP 200 ; l'état de l'Item relu correspond exactement au message :

```
[Coordinateur] DESEQUILIBRE salleB(17.8) / cuisine(25.7) -> ecart 7.9 > seuil 4.0 :
        consigne 21.8 envoyee a cuisine
[Coordinateur->openHAB REST] Batiment_Coordination <- "cuisine: consigne 21.8C
        (desequilibre salleB/cuisine, ecart 7.9C)" (HTTP 200)
$ curl http://localhost:8080/rest/items/Batiment_Coordination/state
cuisine: consigne 21.8C (desequilibre salleB/cuisine, ecart 7.9C)
```

[CAPTURE : log du coordinateur avec la ligne « …REST… (HTTP 200) », et l'Item Batiment_Coordination dans le tableau de bord openHAB]

Tests unitaires. Deux tests JUnit ont été ajoutés dans CoordinateurBatimentTest (via un double FauxOpenHab qui enregistre l'appel au lieu d'émettre une vraie requête HTTP) : l'un vérifie qu'un déséquilibre déclenche bien l'envoi REST sur Batiment_Coordination, l'autre qu'aucun envoi n'a lieu sous le seuil. La suite passe désormais 6 tests (mvn clean test → BUILD SUCCESS, 6/6).

6. Tolérance aux pannes (Tâche 3.1)

Le scénario suivant a été exécuté en conditions réelles (3 simulateurs et 3 agents en processus séparés, coordinateur en boucle continue) :

Étape	Observation
Avant panne	3 pièces équilibrées, coordinateur silencieux (écarts sous le seuil)
Panne (simulateur salleA arrêté)	Aucun effet sur salleB ni cuisine
Détection (~30s plus tard)	agent-salleA journalise : "TIMEOUT détecté : aucun message MQTT depuis plus de 30s. Dernière valeur connue conservée (temperature=25.3)"
Pendant la panne	Coordinateur journalise "salleA en timeout (>30s) - utilisation de la dernière valeur connue" et continue son cycle normalement ; openHAB affiche SalleA_Temperature = UNDEF
Reprise (simulateur relancé)	agent-salleA journalise "Reprise : messages MQTT reçus normalement à nouveau" ; openHAB revient à une valeur numérique

Sortie console réelle (validée le 2026-10-27) — état normal : les 3 pièces publient et affichent des valeurs numériques :

```
$ curl -s .../items/SalleA_Temperature/state   -> 20.3
```

```
$ curl -s .../items/SalleB_Temperature/state   -> 22.0
$ curl -s .../items/Cuisine_Temperature/state  -> 21.2
```

Sortie console réelle (validée le 2026-10-27) — simulateur salleA coupé (kill du PID), état >30s plus tard :

```
[agent-salleA] TIMEOUT détecté : aucun message MQTT depuis plus de 30s.
               Dernière valeur connue conservée (temperature=18.0).
[Coordinateur] salleA en timeout (>30s) - utilisation de la dernière valeur connue
$ curl -s .../items/SalleA_Temperature/state   -> UNDEF    (profil expire)
$ curl -s .../items/SalleB_Temperature/state   -> 20.7     (INCHANGÉE)
$ curl -s .../items/Cuisine_Temperature/state  -> 20.2     (INCHANGÉE)
```

Sortie console réelle (validée le 2026-10-27) — simulateur salleA relancé : reprise automatique :

```
[agent-salleA] Reprise : messages MQTT reçus normalement à nouveau.
$ curl -s .../items/SalleA_Temperature/state   -> 22.5     (de retour numérique)
```

Ce scénario illustre concrètement l'exigence de l'énoncé : "la panne d'une pièce ne doit pas arrêter les autres agents ni le coordinateur", vérifiée à la fois côté agents (aucun impact sur salleB/cuisine) et côté coordinateur (RemoteException gérée par paire, cycle jamais interrompu).

7. Difficultés rencontrées et solutions

Cette section documente des problèmes réels rencontrés pendant le développement, avec leur diagnostic et leur correction — c'est le type d'analyse critique explicitement demandé par le barème du rapport.

7.1 Image Docker openHAB introuvable

Le tag "openhab/openhab:4-debian" utilisé initialement dans docker-compose.yml n'existe plus sur Docker Hub : depuis la sortie d'openHAB 5, seuls des tags de version précise (ex. 4.3.11-debian) restent disponibles pour la branche 4.x. Correction : interrogation de l'API Docker Hub pour lister les tags réellement disponibles et épinglage explicite sur 4.3.11-debian.

7.2 Jar périmé chargé en mémoire par un processus déjà lancé

Après l'ajout de la dépendance org.json et la recompilation du module agent-piece, les 3 agents déjà en cours d'exécution ont continué de planter sur getEtatJson() avec NoClassDefFoundError. Cause : une JVM déjà démarrée garde une référence ouverte vers le contenu du jar tel qu'il était AU MOMENT DU DÉMARRAGE ; recompiler le fichier sur disque ne met pas à jour un processus déjà en cours. Correction : toujours redémarrer les processus agents après chaque recompilation du jar pendant le développement.

7.3 Limitations du moteur de règles DSL openHAB

Une première version des règles utilisait une Map<String,String> initialisée avec l'opérateur -> et la variable magique triggeringItem pour une règle générique couvrant les 3 pièces. Deux erreurs sont apparues : l'inférence de type sur la Map générique échouait silencieusement ("refers to the missing type Object"), et triggeringItem s'est révélé null lorsque plusieurs déclencheurs "Item X changed" étaient combinés par "or". Correction : réécriture des règles avec des blocs explicites par pièce (calqués sur l'exemple fourni dans l'énoncé), plus verbeux mais garanti fiable dans ce moteur restreint.

7.3.1 Note additionnelle : toutes les déclarations globales doivent précéder les règles

Dans un fichier .rules, une déclaration var/val placée ENTRE deux blocs "rule" provoque une erreur de syntaxe ("missing EOF at 'val'"). Toutes les déclarations globales doivent être regroupées en haut du fichier, avant la première règle.

7.3.2 Ambiguïté Java : List.of() avec un seul tableau en argument

Dans les tests JUnit, List.of(new String[]{...}) avec un seul argument tableau est ambigu pour le compilateur : s'agit-il d'un élément unique de type String[], ou du tableau utilisé directement comme varargs (spreading) ? Correction : type witness explicite, List.<String[]>of(...).

7.4 Course entre le simulateur et une valeur forcée manuellement

Pour valider le coordinateur, une température hors norme a été publiée manuellement sur MQTT afin de forcer un déséquilibre. Le simulateur (processus séparé, avec son propre état interne) republie sa propre valeur toutes les 5 secondes indépendamment de toute publication externe, écrasant rapidement la valeur de test. Correction : republication en boucle rapide (toutes les 0.3s) le temps de la vérification. Cette observation illustre un point de conception réel : les capteurs simulés et les commandes externes sont deux sources concurrentes sur le même topic, sans mécanisme d'arbitrage — à discuter comme piste d'amélioration (voir Section 8).

7.5 Incohérence relevée dans l'énoncé

La section "Contrat RMI à respecter" nomme la méthode de résumé d'état getEtatJson(), alors que le squelette de code fourni juste après utilise getEtat(). Le choix a été fait de suivre la description textuelle du contrat (getEtatJson) plutôt que le squelette, puisque c'est elle qui fixe explicitement le nom exigé.

7.6 Panne réelle (non simulée) après plusieurs heures d'exécution

Après plusieurs heures d'exécution continue pendant le développement, les simulateurs de salleA et de cuisine se sont arrêtés silencieusement au même instant (très probablement une coupure réseau transitoire, la machine étant passée en veille), alors que celui de salleB a survécu au même incident. Diagnostic : dans CapteurSimulateur.run(), le bloc try/catch(MqttException) entourait TOUTE la boucle while(true) au lieu d'entourer chaque itération. Une seule publication échouée suffisait donc à faire sortir l'exception de la boucle et à terminer définitivement le thread, même après le retour du réseau — malgré setAutomaticReconnect(true), qui ne protège que la connexion, pas le code appelant qui utilise cette connexion.

Correction : le try/catch(MqttException) a été déplacé À L'INTÉRIEUR de la boucle, autour du seul bloc de publication. Un échec ponctuel est désormais journalisé et le simulateur retente au cycle suivant (5s plus tard) au lieu de s'arrêter. Cet incident, découvert en conditions réelles plutôt que provoqué pour les besoins de la démonstration, illustre que la tolérance aux pannes explicitement demandée pour les AgentPiece (Tâche 3.1) doit en réalité s'appliquer à TOUT composant qui dépend du réseau, y compris les simulateurs eux-mêmes, qui n'étaient pas visés par l'énoncé sur ce point précis.

7.7 Tableau de bord vide : Main UI vs Basic UI

Une fois les Things/Items/Rules en place, la page http://localhost:8080/ (Main UI) restait visuellement vide malgré des données valides confirmées côté REST API. Cause : la Main UI d'openHAB 4 ne génère aucun contenu automatiquement à partir de simples Items — elle exige soit la construction manuelle de pages/widgets dans le navigateur, soit un modèle sémantique (tags Location/Equipment) que nos Items n'avaient pas. Rien à voir avec les données elles-mêmes, qui étaient correctes.

Correction : ajout du binding ui=basic dans addons.cfg (après une première tentative avec l'identifiant incorrect "basicui", corrigé en "basic" après vérification du nom exact de la feature via la console Karaf, feature:list) et

création d'un fichier batiment.sitemap (voir 3.5), qui génère immédiatement un tableau de bord fonctionnel sans aucune configuration manuelle dans le navigateur.

7.8 Confusion openHAB : métadonnée « expire » vs « profile »

Les Items de capteur étaient initialement déclarés avec à la fois profile="expire" et expire="30s,state=UNDEF". Or, dans openHAB, expire est une métadonnée d'Item (espace de noms expire), et non un profile de lien de canal : il n'existe aucun profile nommé « expire » (les profiles comme offset, follow, les transformations, etc., se déclarent entre crochets après le canal). Le profile="expire" était donc une métadonnée morte, au mieux ignorée, au pire source de confusion. Il a été retiré : seule subsiste la forme correcte { channel="…", expire="30s,state=UNDEF" }, qui force bien l'état à UNDEF après 30 s sans mise à jour (Tâche 3.1).

8. Conclusion et pistes d'amélioration

Le projet démontre une plateforme IoT fonctionnelle combinant trois paradigmes de communication distincts (pub/sub MQTT, appel de méthode distant RMI, règles d'automatisation openHAB), avec une tolérance aux pannes vérifiée de bout en bout : un capteur en panne est détecté, signalé, et n'interrompt ni les autres agents ni le coordinateur.

Pistes d'amélioration identifiées mais hors du cadre du projet actuel :

- Arbitrage explicite entre les commandes du coordinateur RMI et les règles DSL openHAB lorsqu'elles ciblent le même actuateur (actuellement, la dernière commande écrite l'emporte, sans coordination).
- Sécurisation MQTT (authentification par mot de passe et ACL par topic, plutôt que allow_anonymous) avant tout déploiement réel.
- Persistance de l'historique des mesures (actuellement seule la dernière valeur est conservée, côté agent comme côté openHAB).

Ce projet a permis de manipuler concrètement trois modèles de communication que le cours présente surtout en théorie, et d'en mesurer les compromis sur un même cas d'usage. Le contraste entre le découplage total de MQTT (les capteurs ignorent qui les écoute) et le couplage fort de RMI (le coordinateur tient une référence directe vers chaque agent) est devenu tangible le jour où les deux modèles ont voulu écrire la même donnée (Section 7.4). La tolérance aux pannes, souvent traitée comme une simple case à cocher, a demandé de distinguer trois niveaux de réponse indépendants — détection côté agent (watchdog), continuité côté coordinateur (dernière valeur connue) et affichage côté openHAB (profil expire) — chacun pouvant fonctionner sans les autres. Sur le plan pratique, les difficultés les plus coûteuses n'ont pas été algorithmiques mais environnementales (image Docker, jar périmé chargé en mémoire, nom exact d'une feature openHAB), ce qui rappelle l'importance d'une procédure de lancement reproductible, consignée en Annexe A. [À personnaliser par l'équipe : répartition du travail entre les membres et apprentissages individuels.]

Annexe A — Instructions de lancement

Prérequis : Java 21, Maven, Docker + Docker Compose.

```
# 1. Infrastructure (Mosquitto + openHAB)
cd smartagent-iot && docker compose up -d

# 2. Build de tous les modules
mvn clean package

# 3. Simulateurs (un processus par pièce, ou tous ensemble sans argument)
java -jar sensor-simulator/target/sensor-simulator.jar salleA
java -jar sensor-simulator/target/sensor-simulator.jar salleB
java -jar sensor-simulator/target/sensor-simulator.jar cuisine

# 4. Agents (un processus par pièce)
java -jar agent-piece/target/agent-piece.jar salleA
java -jar agent-piece/target/agent-piece.jar salleB
java -jar agent-piece/target/agent-piece.jar cuisine

# 5. Coordinateur
java -cp coordinateur/target/coordinateur.jar \
    ca.udes.ift605.smartagent.coordinateur.CoordinateurMain

# Tableau de bord (Basic UI, PAS l'URL racine qui reste vide - voir 3.5/7.7) :
# http://localhost:8080/basicui/app?sitemap=batiment
```

Annexe B — Correspondance avec le barème

Critère (barème)	Points	Section du rapport
MQTT — Pub/Sub	15	4.1, 5.1
openHAB Things	10	4.1, 5.1
openHAB Rules	10	3.4, 4.1, 5.1
Java RMI	20	3.2, 4.2, 5.2
Coordinateur	10	3.3, 4.2, 5.2
Tolérance pannes	10	3.4, 4.3, 6
Rapport PDF	15	(ce document)
Démonstration	10	voir script de démo

Annexe B (complément) — Impact des extensions S2/S3 sur le barème

Les deux extensions renforcent des critères déjà couverts, sans en créer de nouveaux :

- Java RMI (20 pts) : au contrat + 3 serveurs enregistrés s'ajoute une utilisation client de RMI entre agents (consultation pair-à-pair), qui démontre la maîtrise de la dualité serveur/client de RMI.
- Coordinateur (10 pts) et openHAB Rules (10 pts) : la notification REST relie explicitement les décisions du coordinateur au moteur de règles openHAB, au-delà du simple chemin MQTT.
- Tolérance aux pannes (10 pts) : la consultation pair-à-pair et l'appel REST sont tous deux best-effort (gestion NotBound/RemoteException et openHAB absent), cohérents avec la Tâche 3.1.